CONFERENCE TALK · 2026 EDITION

The Ultimate Guide to Browser-Based AI

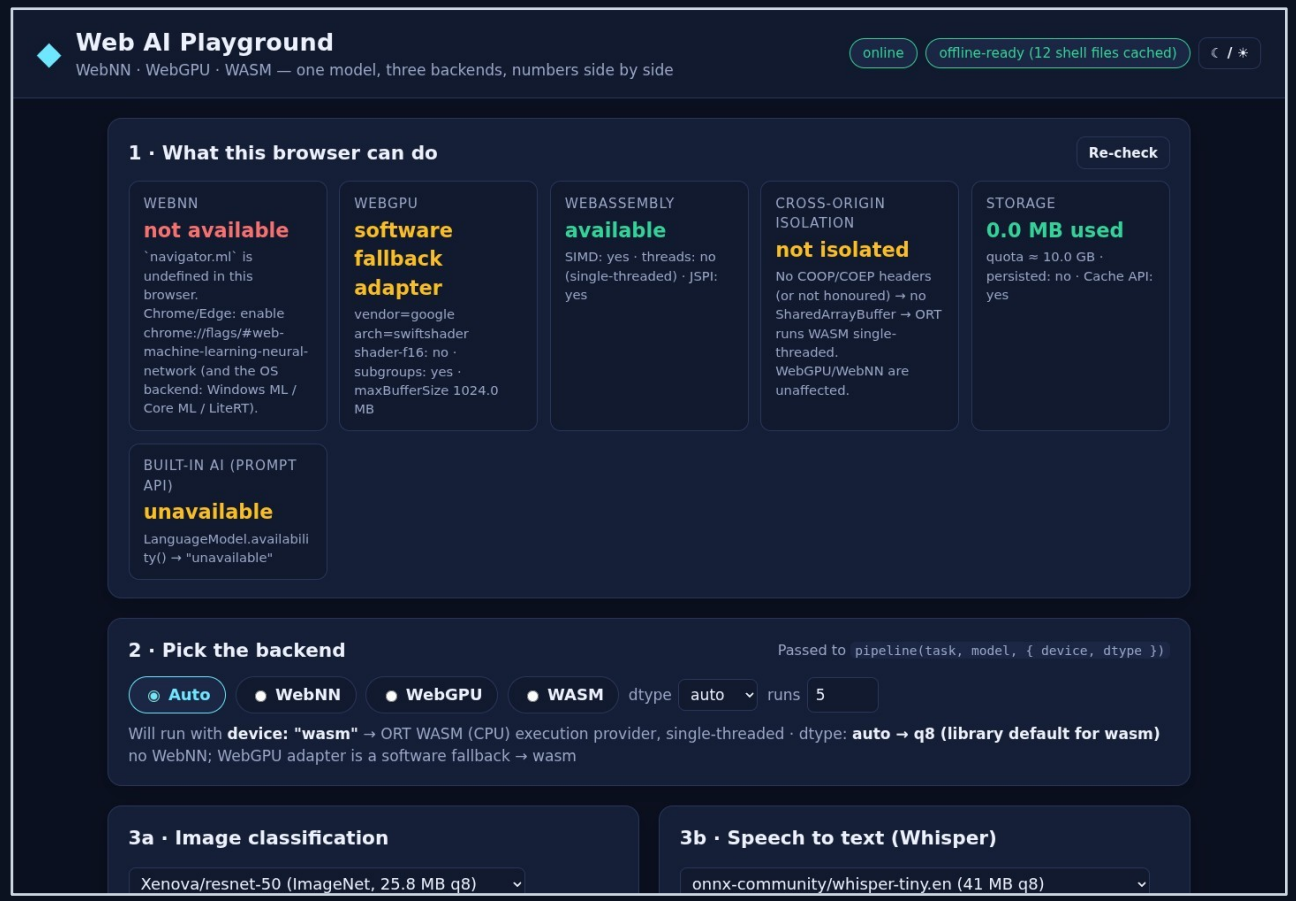
Privacy-First
Offline-Ready
Fully Future-Proof

Maxim Salnikov
AI-Native Solution Engineer at Microsoft · Oslo



LIVE DEMO

Demo 1 · The offline playground



Transformers.js 4.3.1 in a Web Worker · service worker for the app shell · models in Cache Storage

webai-playground.sites.tab.bot

- Wi-Fi off, page loaded once before
- Image classification, speech-to-text
- WASM vs WebGPU, side by side

ABOUT ME

Hi, I'm Maxim

AI-Native Solution Engineer at Microsoft · Oslo

- Building on the web platform since the 90s
- Member of the W3C Web Machine Learning Community Group
- Organizing developer communities and technical conferences
- Speaking, training, blogging: web dev, cloud, generative AI
- Author of “Tools, Not Clicks” (WebMCP) · vibeprogrammer.github.io/webmcp-book
- Newsletter: isainative.substack.com



WHY ON-DEVICE

Why run AI on the user's device?

Privacy

Inference runs locally. Chrome's docs on its built-in model: “No data is sent to Google or any third party when using the model.”

Offline

After the first load, a service worker plus Cache Storage keep the app and its models on the device.

Cost

No per-request cloud bill. The user's CPU, GPU or NPU does the work.

Latency

No round trip. Camera frames, audio and text never leave the tab.

The price: model downloads, very different hardware, and APIs that are not shipped everywhere yet. This talk is about managing that price.

TIMELINE

What changed: 2023 → 2026

2023

WebGPU ships

Chrome & Edge 113 on ChromeOS, macOS, Windows

2024

WebNN dev trial

Chrome 125, behind a flag

2025

Built-in AI + WebGPU everywhere

Translator, Language Detector, Summarizer stable in Chrome 138 · WebGPU in Firefox 141 and Safari 26

2026

Prompt API stable · WebNN in CR

Prompt API on the web in Chrome 148 · WebNN CR Snapshot 22 Jan · Edge 148 ships Translator

GPUs became reachable → NPUs arrived in laptops → browsers started shipping their own models

HARDWARE

CPU, GPU or NPU?

CPU

Always there

- Reached through WebAssembly (SIMD, threads)
- The universal fallback
- Transformers.js default in the browser: wasm

GPU

Massively parallel

- Reached through WebGPU (all four engines) or WebNN
- Best for bigger models, fp16 and LLMs
- Adapters differ: check shader-f16

NPU

Low power, sustained

- Reached only through WebNN → Windows ML / Core ML
- Great for always-on vision and audio
- Web access is behind a flag today

THE MAP FOR THIS TALK

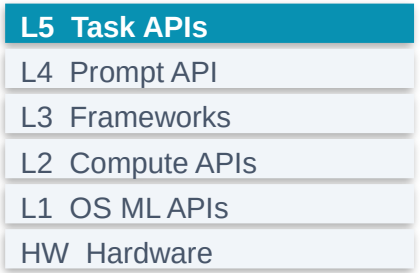
The browser AI stack

L5	**Task APIs**	Translator · Language Detector · Summarizer · Writer / Rewriter · Proofreader	**Stable in Chrome 138+**
L4	**Prompt API**	LanguageModel — Gemini Nano in Chrome, Phi-4-mini in Edge	**Stable in Chrome 148**
L3	**Frameworks**	Transformers.js · ONNX Runtime Web · WebLLM · LiteRT.js	**Libraries: ship today**
L2	**Compute APIs**	WebGPU · WebNN · WebAssembly	**WebGPU everywhere · WebNN flag**
L1	**OS ML APIs**	Windows ML / ONNX Runtime · Core ML · LiteRT	**Used by WebNN**
HW	**Hardware**	CPU · GPU · NPU	

Higher layer = less code, less control. Lower layer = more control, more work.

LAYER 5

L5 · Task APIs: built-in models



Translator

text → another language

expert model

Language Detector

which language is this?

expert model

Summarizer

key points, tldr, teaser, headline

Gemini Nano

Writer / Rewriter

draft or rework text

Gemini Nano

Proofreader

grammar and spelling fixes

Gemini Nano

Same shape for all

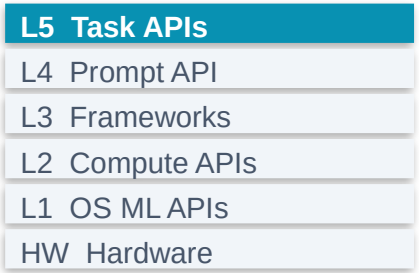
create() → use → destroy()

Translator & Language Detector: desktop only

Foundation-model APIs run on Windows 10/11, macOS 13+, Linux and Chromebook Plus — not on Chrome for Android or iOS.

LAYER 5 · STATUS

Task API status, October 2026



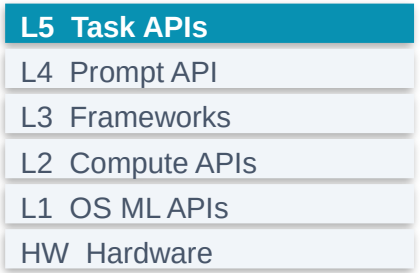
API	Chrome		Edge	
Translator	Stable · 138	desktop only	Stable · 148	145+ languages
Language Detector	Stable · 138	desktop only	Stable · 148	
Summarizer	Stable · 138		Preview	on by default in Canary/Dev
Writer / Rewriter	Flag	origin trial ended	Preview	Canary/Dev, flag
Proofreader	Flag	origin trial ended	—	not listed
Speech, on-device	Stable · 139	processLocally	Preview	Canary/Dev

Firefox and Safari: no Translator, Language Detector or Summarizer (MDN browser-compat-data). Positions: slide 14.

Source: developer.chrome.com/docs/ai/built-in-apis, chromestatus.com, learn.microsoft.com/microsoft-edge — as of 16 Oct 2026

LAYER 5 · CODE

Translate and summarize




```
// Translator — Chrome 138, Edge 148
const translator = await Translator.create({
  sourceLanguage: 'es',
  targetLanguage: 'fr',
});
await translator.translate(
  'Where is the next bus stop, please?');
```

```
// Summarizer — Chrome 138
const summarizer = await Summarizer.create({
  type: 'key-points', // or 'tldr'…
  format: 'markdown',
  length: 'medium',
});
const summary = await summarizer.summarize(
  longText,
  { context: 'For a tech-savvy audience.' });
```

Snippets follow the examples on developer.chrome.com/docs/ai. Check `availability()` first — the model may need a download.

LAYER 4

L4 · Prompt API in four steps


```
if (!('LanguageModel' in self)) return useFallback();
const status = await LanguageModel.availability();
if (status === 'unavailable') return useFallback();
const session = await LanguageModel.create({
  monitor(m) {
    m.addEventListener('downloadprogress',
      (e) => showProgress(e.loaded));
  },
});
for await (const c of session.promptStreaming(text))
  output.append(c);
```

1. **Detect**
   'LanguageModel' in self — never throws
2. **Ask**
   availability() with the same options you'll use
3. **Create**
   create() may trigger the model download
4. **Prompt**
   prompt() or promptStreaming(); destroy() when done

LAYER 4 · STATUS

Prompt API: status and hardware

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

Chrome	**Stable · 148**	on the web · Extensions since 138
Edge	**Preview**	Canary/Dev from 138, Phi-4-mini
Sampling	**Origin trial**	temperature/topK on the web: OT from Chrome 148

Chrome (Gemini Nano)

≥ 22 GB free disk on the profile volume

GPU with more than 4 GB VRAM, or CPU with 16 GB RAM and 4 cores

Windows 10/11, macOS 13+, Linux, Chromebook Plus

Audio input needs a GPU

Edge (Phi-4-mini)

≥ 20 GB free disk

GPU with 5.5 GB VRAM or more

Windows 10/11, macOS 13.3+

Aion-1.0-Instruct preview from Edge 150 for weaker devices

Source: chromestatus.com, developer.chrome.com/docs/ai, learn.microsoft.com (Edge Prompt API) — as of 16 Oct 2026

LAYER 4 · EDGE

Same API, different model

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

Chrome

Gemini Nano

Stable on the web in Chrome 148

Text, image and audio input

Edge

Phi-4-mini (4B parameters)

Dev preview, Canary/Dev 138+

Aion-1.0-Instruct preview in Edge 150

```
// same code, Chrome and Edge
const s = await LanguageModel
  .create();
const out = await s.prompt(
  'Summarize: ' + text);
```

Write once, model varies.

Answers, quality, speed and languages differ by browser. Test prompts on both and keep outputs short and structured. Interoperability is exactly the concern Mozilla and WebKit raised.

STANDARDS, HONESTLY

Who's on board? Standards and browser positions

API	Where it's specified	Mozilla	WebKit
WebNN	W3C Working Group · Candidate Rec.	Positive	No position
Prompt API	WebML Community Group	Negative	Oppose
Writing Assistance	WebML Community Group	Negative	Open
Language Detector	WebML Community Group	Open	—
WebMCP	WebML Community Group	Neutral	Oppose

Community Group = incubation. Only WebNN is on the Recommendation track (target: Q1 2027).

Source: W3C WebML WG/CG charters, github.com/mozilla/standards-positions, github.com/WebKit/standards-positions — as of 16 Oct 2026

LAYER 3

L3 · Frameworks: bring a model

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

Transformers.js 4.3.1

Hugging Face pipelines on ONNX Runtime Web

wasm · webgpu · webnn

4,272 models on the Hub

ONNX Runtime Web 1.30

Run any ONNX model

EPs: wasm, webgpu, webnn

The engine under Transformers.js

WebLLM 0.2.86

LLM chat engine (MLC)

WebGPU only

Llama-3.2-1B q4f16 ≈ 879 MB

LiteRT.js 2.5.3

Google's LiteRT (ex-TFLite) for the web

WebGPU / WASM

+ MediaPipe tasks-genai 0.10.29

TensorFlow.js: last release 4.22.0 in October 2024 — maintenance mode in practice. Start new work elsewhere.

Source: npm registry, GitHub releases — versions as of 16 Oct 2026

Transformers.js 4.3: what's new

4.3.1
released 6 Oct 2026

4,272
models on the Hub (was 2.7K+)

~200
architectures tested on the new WebGPU runtime

- v4: WebGPU runtime in ONNX Runtime rewritten in C++
- ModelRegistry, standalone Tokenizers.js, monorepo
- Models cached in Cache Storage by default
- dtype per model file, e.g. encoder fp16 + decoder q4

device and dtype

device: `wasm · webgpu · webnn · webnn-npu · webnn-gpu · webnn-cpu · auto`

dtype: `fp32 · fp16 · q8 · int8 · uint8 · q4 · bnb4 · q4f16`

Default: q8 on wasm, fp32 elsewhere. Gotcha: plain 'webnn' means deviceType cpu.

Source: github.com/huggingface/transformers.js releases 4.0.0 and 4.3.1, src/utils/devices.js, dtypes.js — read 16 Oct 2026

LAYER 3 · CODE

One option switches the backend

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

Transformers.js — condensed from the demo's inference-worker.js

```
import { pipeline, env }
  from '@huggingface/transformers';  // v4.3.1
env.useBrowserCache = true;  // Cache Storage

const classify = await pipeline(
  'image-classification', 'webnn/mobilenet-v2', {
    device: 'webnn-npu',   // or 'webgpu', 'wasm'
    dtype: 'fp16',
    progress_callback: (p) => updateBar(p),
  });
const out = await classify(image, { top_k: 5 });
```

ORT Web WebNN execution provider

```
// One layer down: ONNX Runtime Web
const session = await
  ort.InferenceSession.create(
  'model.onnx', {
    executionProviders: [{
      name: 'webnn',
      deviceType: 'npu', // cpu|gpu
      powerPreference: 'default',
    }],
  });
```

Transformers.js turns 'webnn-npu' into exactly this executionProviders entry.

L2 · WebGPU: in all four engines

Chrome / Edge	**Stable · 113**	ChromeOS, macOS, Windows (2023)
Chrome Android	**Stable · 121**	Android 12+, Qualcomm / ARM GPUs
Chrome Linux	**Rolling out · 144**	Intel Gen12+ first, AMD / NVIDIA planned
Firefox	**Stable · 141**	Windows; macOS on Apple Silicon from 145/147; not Linux or Android yet
Safari	**Stable · 26.0**	macOS, iOS, iPadOS, visionOS

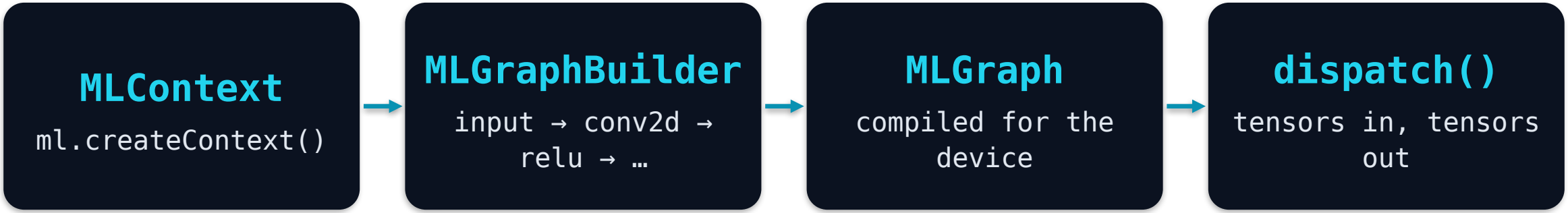
For ML: `shader-f16` (Chrome 120) and subgroups (Chrome 134) are optional per adapter — check `adapter.features.has('shader-f16')` before choosing fp16 weights.

Source: chromestatus.com, web.dev/blog/webgpu-supported-major-browsers, webkit.org, MDN browser-compat-data — as of 16 Oct 2026

LAYER 2 · WEBNN

L2 · WebNN: neural nets for the web

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware



- Graph-level API: the browser hands the whole graph to the OS ML stack — that's how it reaches the NPU
- Tensors stay on the device between calls; interop with WebGPU buffers
- You rarely write it by hand: ONNX Runtime Web and LiteRT.js use it as a backend
- Available in windows and workers, secure contexts only

LAYER 2 · WEBNN STATUS

WebNN in Oct 2026: behind a flag

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

Spec

W3C Candidate Recommendation

CR Snapshot: 22 Jan 2026

Latest CR Draft: 10 Oct 2026

Charter target: Recommendation in Q1 2027

Chrome	**Flag**	Dev trial since 125. Origin trial pulled 27 Mar 2026
Edge	**Flag**	Origin trial expired 11 Aug 2026
Firefox	**Not supported**	Mozilla position: positive
Safari	**Not supported**	WebKit: no position yet

Install guide: webnn.io

Try it in Chrome or Edge Canary (Windows ML: Windows 11 24H2+)
`about://flags/#web-machine-learning-neural-network`

Source: api.w3.org, chromestatus.com/feature/5176273954144256, blink-dev Intent to Experiment, Edge origin trials, webnn.io — as of 16 Oct 2026

LAYER 2 · WEBNN CODE

Device selection: spec vs. Chromium

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

SPEC: no deviceType any more

```
// The spec (CR Draft, Oct 2026)
const ctx = await navigator.ml
  .createContext({
    powerPreference: 'high-performance',
    accelerated: true,
  });
ctx.accelerated; // got acceleration?
```

CHROMIUM TODAY (behind the flag)

```
// What Chromium implements today
const ctx = await navigator.ml
  .createContext({ deviceType: 'npu' });
// deviceType: 'cpu' | 'gpu' | 'npu'
// ORT Web + Transformers.js pass it too
```

Feature-detect both shapes, never crash — as the playground does

```
if ('ml' in navigator) {
  try { ctx = await navigator.ml.createContext({ deviceType: 'npu' }); }
  catch (e) { showReason(`${e.name}: ${e.message}`); }  // fall back, show why
}
```

LAYER 1

L1 · Under WebNN: the OS ML stack

L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

WebNN in Chromium

Windows 11 24H2+

Windows ML / ONNX Runtime

- NPU and GPU via vendor EPs
- EP order: NPU > GPU > CPU
- WebGPU EP ranked before DirectML
- CPU via LiteRT / XNNPACK

macOS 14.4+, Apple Silicon

Core ML

- deviceType 'npu' → CPU + Neural Engine
- otherwise all compute units

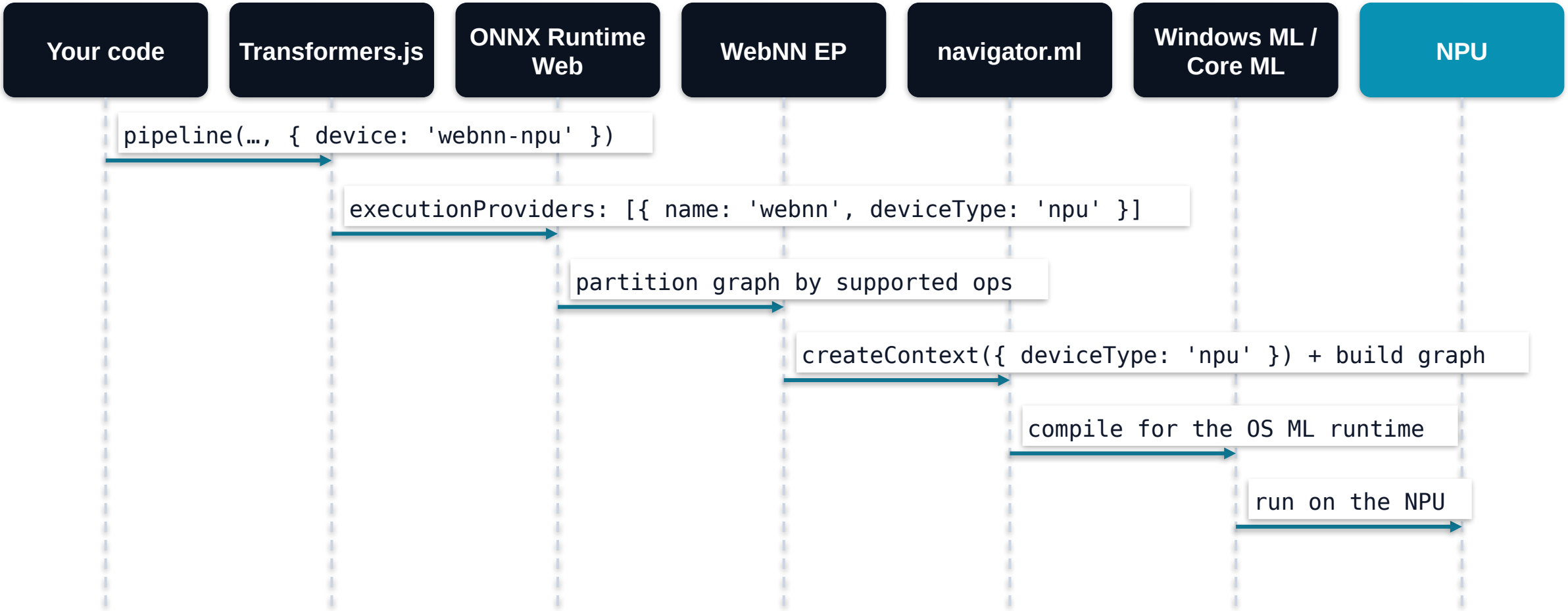
ChromeOS · Linux · Android

LiteRT (ex-TFLite)

- Also the fallback everywhere
- NPU requests run on CPU (XNNPACK) for now

Source: webnn.io/en/api-reference/browser-compatibility/deviceselection and /windowsml, webnn.io/en/faq/hardware — as of 16 Oct 2026

How 'webnn-npu' reaches the NPU



Ops the WebNN EP can't run fall back to WASM — that's why op coverage and static shapes matter.

DEEP DIVE · HONEST ANSWER

Which models run on WebNN today?

Built for WebNN

Hugging Face “webnn” org: 42 models, 5 Spaces

e.g. mobilenet-v2, whisper-base-webnn, yolo12n, depth-anything, segment-anything, Phi-4-mini-instruct-onnx-webnn

What makes a model work

Static input shapes (freeDimensionOverrides)

Ops the WebNN EP supports

fp16 weights for GPU / NPU

In the playground

`webnn/mobilenet-v2` fp16, 7 MB — exported for the WebNN EP: the safe bet on stage

`Xenova/resnet-50` · `whisper-tiny.en` · `SmolLM2-135M` — try them on WebNN live; no maintained compatibility list exists

LIVE DEMO

Demo 2 · WebNN × Transformers.js on the NPU

1. Chrome / Edge Canary, WebNN flag on
2. Capability panel: which contexts open
3. Device: WebNN → NPU, model mobilenet-v2 fp16
4. Task Manager → NPU graph moves
5. Compare WASM · WebGPU · WebNN rows
6. Bonus: Whisper on the JFK clip



webai-playground.sites.tab.bot

Measured on this laptop, live. The NPU row is filled in on stage.

MEASURED, NOT PROMISED

Demo 2 · the numbers

Model	Backend	dtype	Download	Load	Per run (avg)
ResNet-50	WASM	q8	24.7 MB	2.97 s	256 ms (186 ms local)
MobileNet-v2 (webnn org)	WASM	fp16	6.8 MB	1.35 s	85 ms
ResNet-50	WebGPU*	fp32	97.5 MB	3.58 s	8.4 s (software GPU)
Whisper tiny.en · 11 s clip	WASM	q8	41.5 MB	2.6 s	2.0 s
SmolLM2-135M-Instruct	WASM	q8	132.8 MB	4.6 s	18.3 tok/s (24.4 local)
WebNN · NPU	measure on stage →	_____	_____	_____	

LAYER 4 · BEYOND TEXT

Prompt API: images, audio, JSON

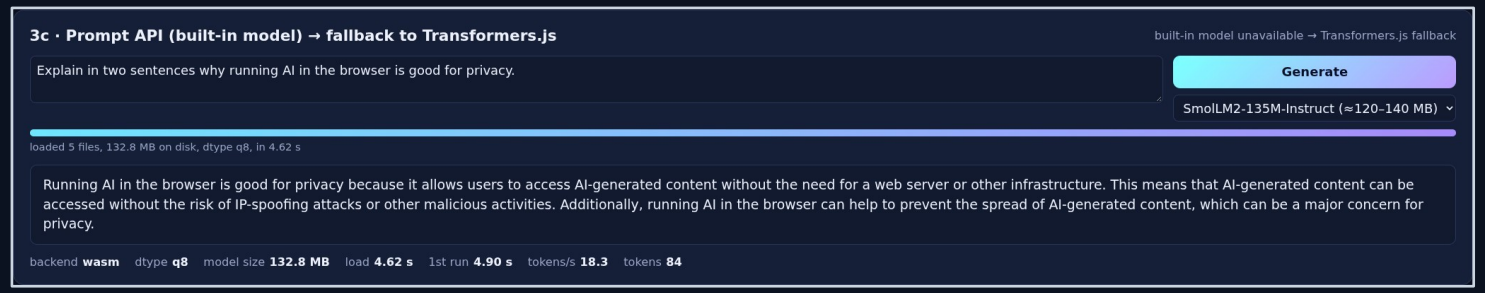
L5 Task APIs
L4 Prompt API
L3 Frameworks
L2 Compute APIs
L1 OS ML APIs
HW Hardware

```
const session = await LanguageModel.create({
  expectedInputs: [{ type: 'image' }] });
const schema = { type: 'object',
  properties: { label: { type: 'string' } } };
const answer = await session.prompt([{
  role: 'user',
  content: [
    { type: 'text', value: 'What is in this photo?' },
    { type: 'image', value: photoBlob },
  ],
}], { responseConstraint: schema });
```

- Inputs: text, image, audio (shipped in Chrome 148)
- Output: text only
- `responseConstraint`: JSON Schema or regex
- Audio input needs a GPU
- Declare expectedInputs up front so availability() is accurate

LIVE DEMO

Demo 3 · Prompt API with a fallback



`'LanguageModel' in self` and availability() ≠ 'unavailable'

yes → built-in model create() + progress, then stream

no → Transformers.js SmolLM2-135M-Instruct q8, in a worker

webai-playground.sites.tab.bot

Same UI, different engine.

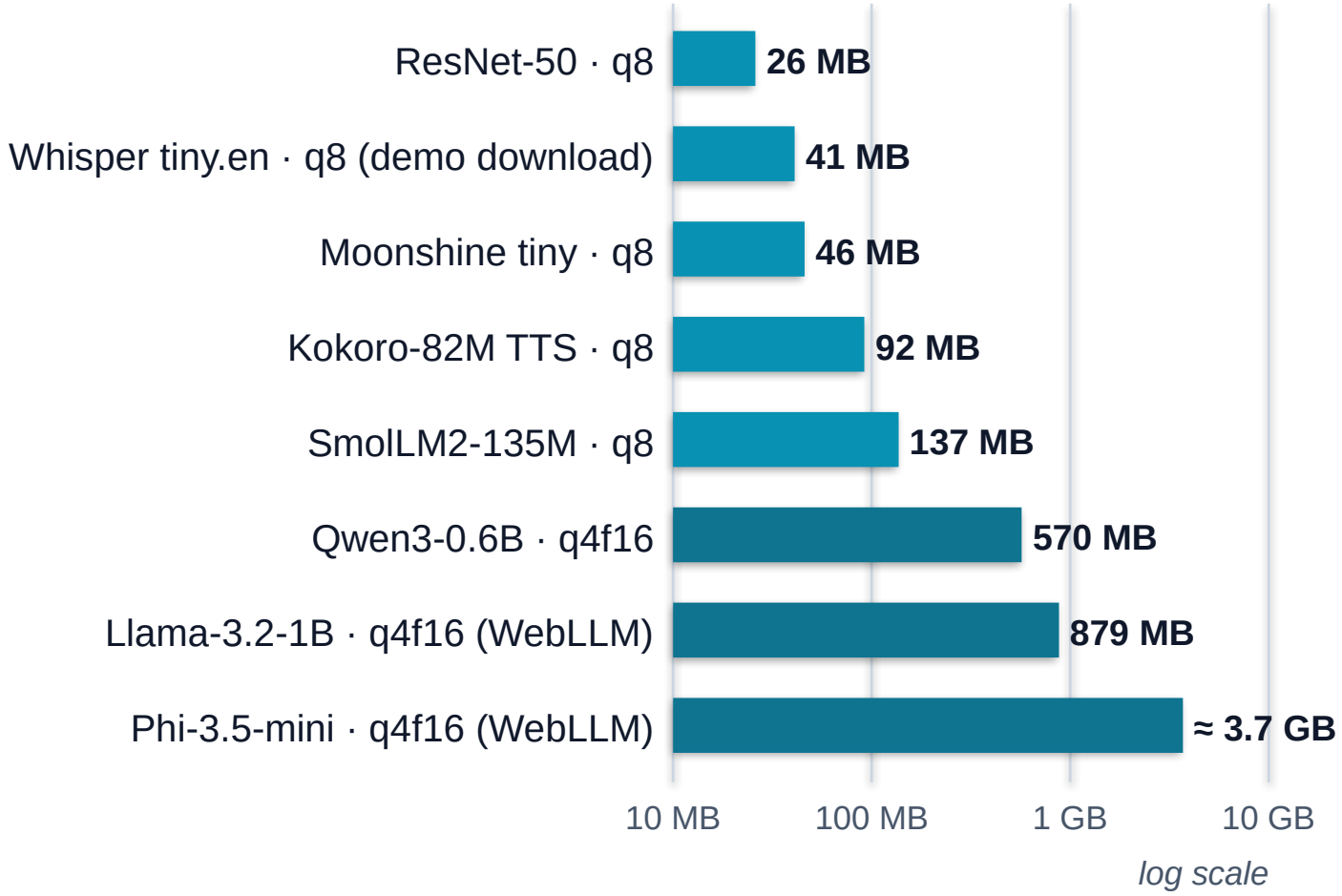
DECISION MATRIX

Which layer for which job?

Job	Start with	Fallback
Translate, detect language	**L5 Translator / Language Detector**	Transformers.js translation model
Summarize, rewrite, proofread	**L5 Summarizer, Writer, Rewriter**	L4 Prompt API → small LLM
Chat, extraction, JSON output	**L4 Prompt API**	Transformers.js / WebLLM, or cloud
Vision: classify, detect	**L3 Transformers.js**	Same model on WASM
Speech-to-text	**Web Speech, processLocally**	Whisper / Moonshine in Transformers.js
Your own ONNX model	**L3 ONNX Runtime Web**	WASM execution provider

Rule: take the highest layer that does the job, and always have the next one down ready.

Models are big. Plan the bytes.



Where they live

Cache Storage: Transformers.js default

OPFS or your own store: env.useCustomCache

Chromium: up to 60 % of disk per origin

Safari apps ≈ 60 %, embedded WebKit ≈ 15 %

Ask for navigator.storage.persist()

Source: Hugging Face Hub file listings, WebLLM config.ts, MDN Storage quotas and eviction criteria — read 16 Oct 2026

Offline: app shell + model cache

Service worker (condensed from the demo's sw.js)

```
// sw.js — model files from the Hub
if (url.pathname.includes('/resolve/')) {
  e.respondWith(fetch(e.request).catch(async () =>
    (await caches.match(e.request)) ||
    new Response('', { status: 404 })));
  return;
}
// app shell: cache-first, refresh in background
const hit = await c.match(e.request,
  { ignoreSearch: true });
return hit || (await fetchP) ||
  new Response('offline', { status: 503 });
```

inference-worker.js

```
env.useBrowserCache = true;
// → 'transformers-cache'
```

The bug we hit

Optional files that 404 online were never cached, so offline the fetch threw.

Fix: answer a synthetic 404 when there's no cached copy.

FUTURE-PROOF

The fallback chain



Pattern — every check is safe in every browser

```
async function pickBackend() {
  if ('LanguageModel' in self &&
      (await LanguageModel.availability()) !== 'unavailable') return 'built-in';
  if ('ml' in navigator) {
    try { await navigator.ml.createContext({ deviceType: 'npu' }); return 'webnn-npu'; }
    catch { /* no NPU context: keep going */ }
  }
  if (await navigator.gpu?.requestAdapter()) return 'webgpu';
  return 'wasm';   // works everywhere; offer the cloud only if the device is too slow
}
```

THE NEXT LAYER

What's next: WebMCP — tools, not clicks

Your page exposes typed tools — JS functions with descriptions and JSON-Schema inputs — that the browser's AI agent can call in the user's session.

```
document.modelContext.registerTool({
  name: 'add_to_cart',
  description: 'Add a product to the cart',
  inputSchema: { type: 'object',
    properties: { sku: { type: 'string' } } },
  execute: async ({ sku }) => cart.add(sku),
});
```

Chrome OT · M149–M162

W3C CG draft · 16 Oct 2026

Mozilla: neutral · WebKit: oppose

My book:

Tools, Not Clicks

Building Agent-Ready Websites with WebMCP

vibeprogrammer.github.io/webmcp-book

Source: webmachinelearning.github.io/webmcp, chromestatus.com/feature/5117755740913664, standards-positions repos — as of 16 Oct 2026

TAKEAWAYS

Three things to take home

1 **Pick the highest layer that does the job**

Task APIs → Prompt API → frameworks. Less code, fewer bytes.

2 **Detect, never assume**

availability(), createContext(), requestAdapter(). WASM always works.

3 **Ship it offline**

Service worker + Cache Storage. Treat models as assets. Test with the network off.

WebNN today: flag in Chrome & Edge, spec in CR, Recommendation targeted for Q1 2027. Build the path now.

LET'S CONNECT

Thank you!

LinkedIn

this deck + all links

webai-playground.sites.tab.bot

Try the playground

Newsletter

isainative.substack.com

Book

Tools, Not Clicks

vibeprogrammer.github.io/webmcp-book

Maxim Salnikov

AI-Native Solution Engineer at Microsoft